

AutoShorts AI

API Documentation

Error Handling + API Validation

Error Handling

All errors return a JSON response with a human-readable message. No raw exceptions are exposed to the client.

Invalid Video Format

Triggered when the uploaded file has an unsupported or missing extension.

```
ALLOWED_EXTENSIONS = {'mp4', 'mov', 'avi', 'mkv', 'webm'}

if '.' not in filename:
    return jsonify({"success": False,
                    "message": "Invalid file. Please upload mp4, mov, avi, mkv or webm."}), 400

ext = filename.rsplit('.', 1)[-1].lower()
if ext not in ALLOWED_EXTENSIONS:
    return jsonify({"success": False,
                    "message": f"Format '{ext}' not supported."}), 400
```

```
{
  "success": false,
  "message": "Format '.pdf' not supported. Please upload mp4, mov, avi, mkv or webm."
}
```

Status: 400 Bad Request

Empty Upload

Triggered when no file is attached to the request, or the file field is empty.

```
if 'file' not in request.files:
    return jsonify({"success": False, "message": "No video uploaded."}), 400

if uploaded_file.filename == '':
    return jsonify({"success": False, "message": "No file selected."}), 400
```

```
{
  "success": false,
```

```
    "message": "No video uploaded."
}
```

Status: 400 Bad Request

Missing FFmpeg

Checked at the start of every request. If FFmpeg is not installed, processing cannot continue.

```
def check_ffmpeg():
    try:
        subprocess.run(['ffmpeg', '-version'],
                        stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL, check=True)
        return True
    except (subprocess.CalledProcessError, FileNotFoundError):
        return False

if not check_ffmpeg():
    return jsonify({"success": False,
                  "message": "FFmpeg is not installed on this server."}), 500
```

```
{
  "success": false,
  "message": "FFmpeg is not installed on this server."
}
```

Status: 500 Internal Server Error

Missing Transcript

Triggered when transcription completes but no speech segments are found in the video.

```
segments = transcript_data.get('segments', [])
if not segments:
    update_progress(progress_data["progress"], "No speech detected in video.",
False)
    return jsonify({"success": False,
                  "message": "No speech detected. Please upload a video with clear audio."}),
422

except FileNotFoundError as e:
    return jsonify({"success": False, "message": f"Video file missing: {e}"}), 404

except RuntimeError as e:
    return jsonify({"success": False, "message": f"Transcription error: {e}"}), 500

except Exception:
    return jsonify({"success": False,
                  "message": "Transcription failed. Please ensure video has clear audio."}),
500
```

```
{
  "success": false,
  "message": "No speech detected. Please upload a video with clear audio."
}
```

Status: 422 Unprocessable Entity

Rendering Failure

Triggered when FFmpeg fails to render a clip, or the output file is not found after rendering.

```
try:
    render_clip(file_path, segment['start'], segment['end'], out_path, cw, ch)
except Exception:
    return jsonify({"success": False,
                    "message": f"Rendering failed for clip {index}. Please try again."}), 500

if not os.path.exists(out_path):
    return jsonify({"success": False,
                    "message": f"Clip {index} was not generated."}), 500
```

```
{
  "success": false,
  "message": "Rendering failed for clip 2. Please try again."
}
```

Status: 500 Internal Server Error

Server Errors

Catch-all for any unhandled exception. Flask error handlers cover all routes globally.

```
except Exception as e:
    print(f'Error: {e}')
    update_progress(progress_data["progress"], "Something went wrong.", False)
    return jsonify({"success": False,
                    "message": "An unexpected error occurred. Please try again."}), 500

@app.errorhandler(404)
def not_found(e):
    return jsonify({"success": False, "message": "Resource not found."}), 404

@app.errorhandler(500)
def server_error(e):
    return jsonify({"success": False, "message": "Internal server error."}), 500

@app.errorhandler(413)
def file_too_large(e):
```

```
return jsonify({"success": False, "message": "File too large. Maximum
500MB."}), 413
```

```
{
  "success": false,
  "message": "An unexpected error occurred. Please try again."
}
```

Status: 500 Internal Server Error

Friendly Messages vs Raw 500 Errors

Condition	Without Handling	With Handling
Unsupported format	500 Internal Server Error	"Format '.pdf' not supported."
No file uploaded	Raw Flask 400 page	"No video uploaded."
FFmpeg missing	Application crash	"FFmpeg is not installed on this server."
No speech in video	Silent failure	"No speech detected. Please upload a video with clear audio."
Clip render fails	Unhandled exception	"Rendering failed for clip 2. Please try again."
File over 500MB	Flask 413 HTML page	"File too large. Maximum 500MB."
Unknown crash	Raw 500 HTML page	"An unexpected error occurred. Please try again."

API Validation

File Type Validation

Only these extensions are accepted: mp4, mov, avi, mkv, webm.

```
ALLOWED_EXTENSIONS = {'mp4', 'mov', 'avi', 'mkv', 'webm'}

ext = filename.rsplit('.', 1)[-1].lower()
if ext not in ALLOWED_EXTENSIONS:
    return jsonify({"success": False,
                    "message": f"Format '{ext}' not supported."}), 400
```

Input Filename	Result
video.mp4	Accepted
clip.mov	Accepted
document.pdf	Rejected — 400
image.png	Rejected — 400
noextension	Rejected — 400

File Size Validation

Maximum size is 500MB. The file is deleted from disk before returning the error.

```
MAX_FILE_SIZE_MB = 500

size_mb = os.path.getsize(file_path) / (1024 * 1024)
if size_mb > MAX_FILE_SIZE_MB:
    os.remove(file_path)
    return jsonify({"success": False,
                    "message": f"File too large ({size_mb:.1f}MB). Maximum allowed size is
{MAX_FILE_SIZE_MB}MB."}), 400
```

```
{
  "success": false,
  "message": "File too large (612.3MB). Maximum allowed size is 500MB."
}
```

Status: 400 Bad Request

Missing Fields

file is required. model_size and max_clips are optional with safe defaults.

```
if 'file' not in request.files:
    return jsonify({"success": False, "message": "No video uploaded."}), 400

model_size = request.form.get('model_size', 'tiny')

try:
    max_clips = int(request.form.get('max_clips', 3))
except (ValueError, TypeError):
    max_clips = 3
```

Field	Required	Default
file	Yes	—

Field	Required	Default
model_size	No	"tiny"
max_clips	No	3

Empty Requests

```
if 'file' not in request.files:
    return jsonify({"success": False, "message": "No video uploaded."}), 400

if uploaded_file.filename == '':
    return jsonify({"success": False, "message": "No file selected."}), 400
```

Invalid API Requests

Requests to unknown endpoints or non-existent clip files return JSON, not HTML error pages.

```
@app.route('/clips/<filename>')
def serve_clip(filename):
    file_path = os.path.join(OUTPUT_FOLDER, filename)
    if os.path.exists(file_path):
        return send_file(file_path, mimetype='video/mp4')
    return jsonify({'success': False, 'message': 'Clip not found.'}), 404

@app.errorhandler(404)
def not_found(e):
    return jsonify({"success": False, "message": "Resource not found."}), 404
```

```
{
  "success": false,
  "message": "Clip not found."
}
```

Status: 404 Not Found

Flask Exception Handling

Two levels are used:

Route-level try/except blocks inside `/api/transcribe` catch errors per pipeline stage and return a specific message for each failure point — transcription, scoring, selection, rendering.

Global `@app.errorhandler` decorators intercept HTTP exceptions across all routes, ensuring every error returns JSON regardless of where it originates.

HTTP Status Codes

Code	Meaning	When Used
200	OK	Processing completed, clips returned
400	Bad Request	Wrong format, missing file, file too large
404	Not Found	Unknown endpoint, clip not on disk
413	Payload Too Large	Upload exceeds server limit
422	Unprocessable Entity	No speech detected, no clips selected
500	Internal Server Error	FFmpeg missing, render error, unhandled exception

JSON Response Standards

All responses use the same envelope. success and message are always present.

Success Response

```
{
  "success": true,
  "message": "Processing completed!",
  "clips": [
    {
      "url": "/clips/clip_001.mp4",
      "start": 12.4,
      "end": 72.1,
      "score": 0.87
    }
  ]
}
```

Error Response

```
{
  "success": false,
  "message": "No video uploaded."
}
```

Progress Response (/api/progress)

```
{
  "progress": 70,
  "status": "Selecting best clips...",
  "success": true
}
```

Response Contract

Field	Type	Always Present	Notes
success	boolean	Yes	true on 2xx, false on 4xx/5xx
message	string	Yes	Human-readable, never a raw exception
clips	array	No	Only on 200 from /api/transcribe
Content-Type	header	Yes	Always application/json